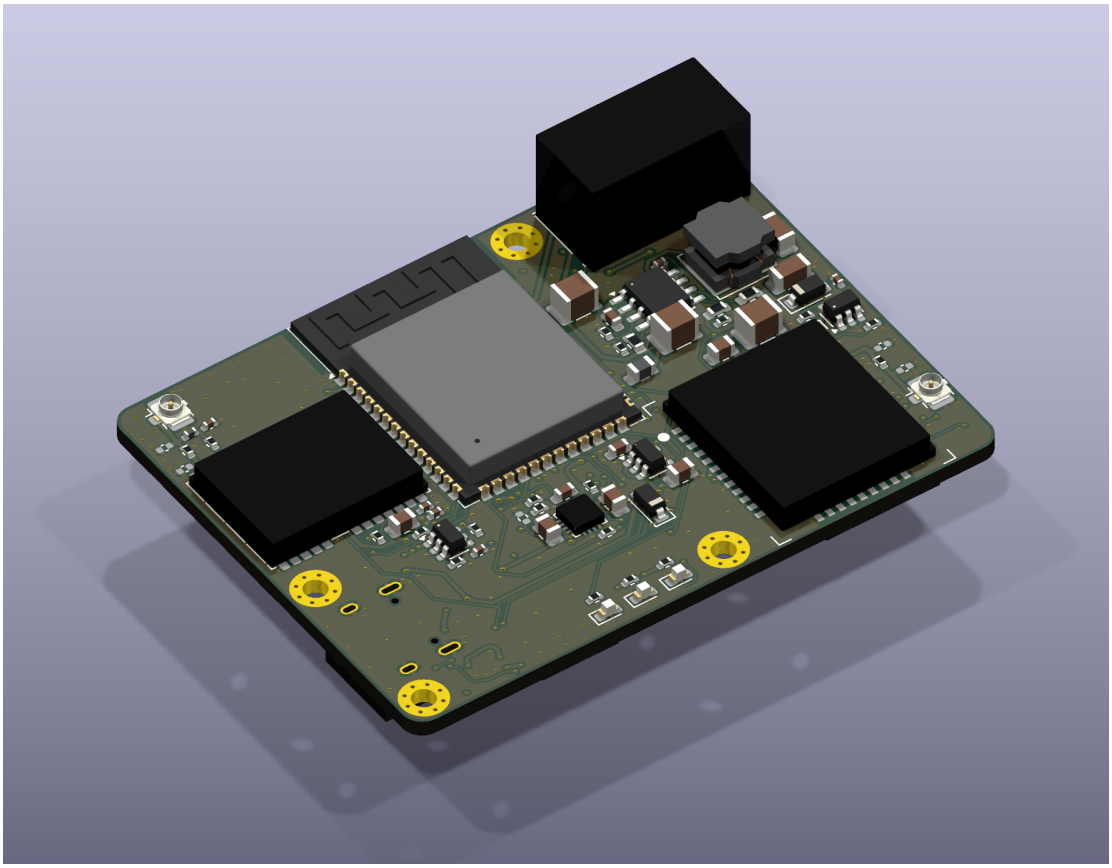
Motorcycle GPS Tracker

How it works, how it was built, and what it is used for



The finished CRL-TRK-2W-01 board (60 × 45 mm), 3D view generated from the KiCad design.

What	Where	Power	Status
GPS + NavIC tracker with LTE-M/NB-IoT, motion sensor, SOS input and starter-cut relay output	India, AIS-140 commercial fleets; motorcycles and scooters (12 V and 24 V vehicles)	9–36 V from the bike + 18650 backup pack (≈ 40 h of tracking after a wire cut)	Rev A0 design complete: ERC/DRC clean, Gerbers, BOM and pick-and-place ready for prototypes

Who should read this: fleet owners, installers, firmware developers, hardware and production engineers. Each page starts with a plain-language summary; the details follow below it.

Contents

#	Section	For
1	The big picture: what the tracker does	everyone
2	Inside the box: block diagram	everyone, engineers
3	Use cases: real situations and what happens	owners, fleet managers, sales
4	Operating workflow: the device's modes	owners, firmware, support
5	Step-by-step flows for the key alerts	firmware, control room, support
6	Installation and wiring	installers
7	Design workflow: how the PCB was made	hardware engineers
8	Production and test workflow	factory, quality
9	Which file do I need?	everyone
10	Glossary and open items	everyone

> **In one sentence:** a small box under the seat that always knows where the bike is, tells the owner and the control room over the mobile network, raises an alarm if the bike is moved, crashed, has its power cut or the rider presses SOS, and can stop the engine from starting on command.

1. The big picture

The tracker reads the satellites to know its position, adds what it feels (motion, crash, ignition, battery) and sends a short message over LTE-M / NB-IoT. That is the low-power 4G variant made for machines. A cloud server stores the messages, applies the rules (geofence, alarms) and shows the result in an app and, for AIS-140 vehicles, in the state control room. Commands such as “immobilise” or “update firmware” travel back the same way.

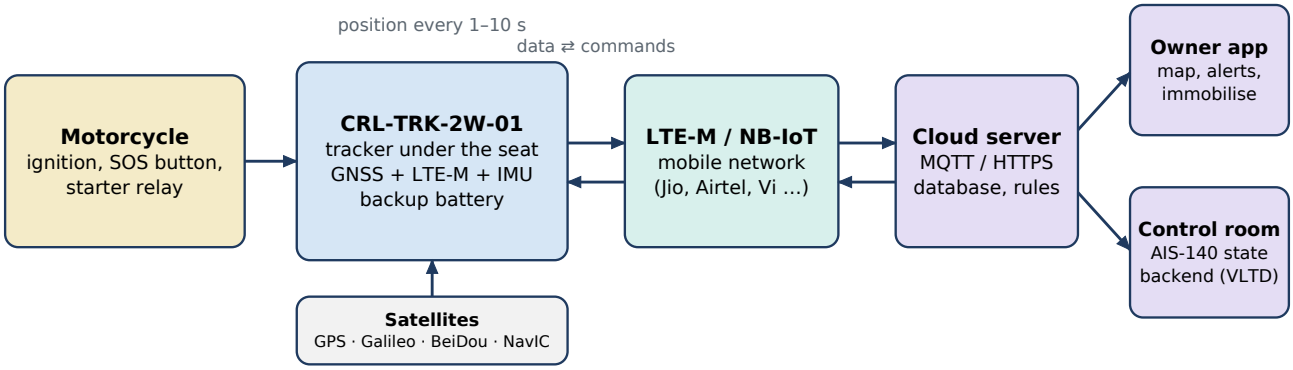


Figure 1: from the bike to the people who need the information.

2. Inside the box

Power comes from the vehicle and, when that is missing, from the backup pack. Everything is protected against the electrical abuse of a motorcycle: load dumps, reversed wires and spikes. A single microcontroller runs the rules and talks to three specialist chips.

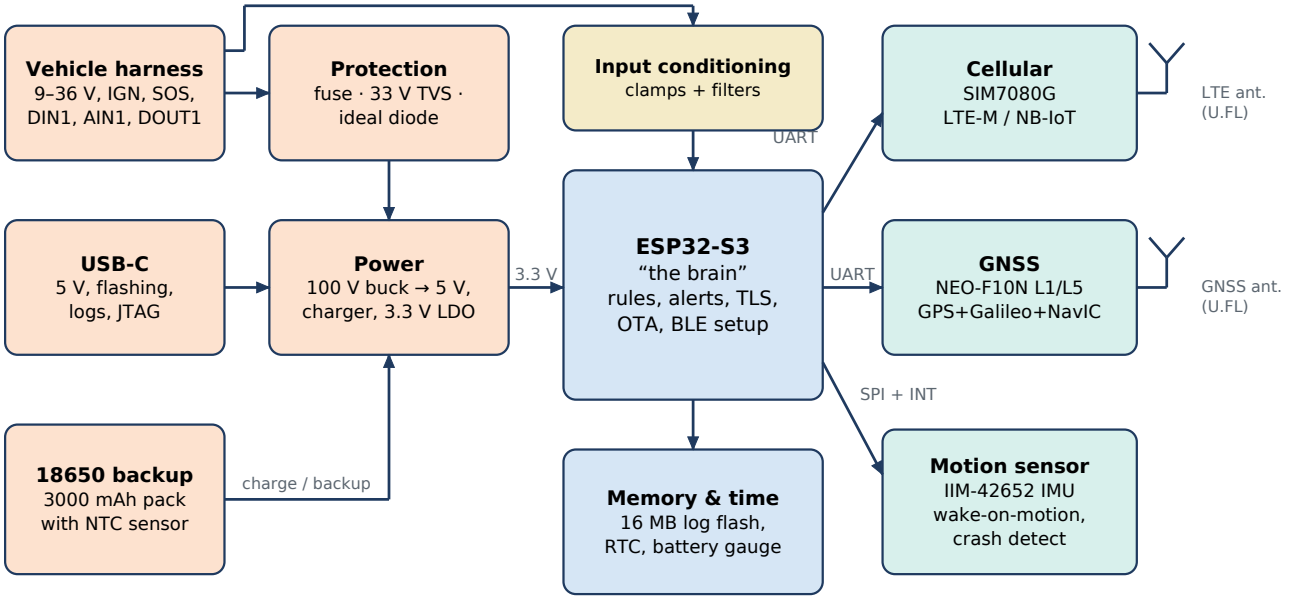


Figure 2: functional block diagram (orange = power, blue = control, green = radios/sensors).

What each block does

Block	Main part	Job, in plain words
Protection	fuse, SMBJ33CA TVS, LM74700-Q1 + 60 V MOSFET	Survives a reversed battery (−33 V), load-dump spikes and jump starts without blowing up.
Power	LM5164-Q1, BQ24074, TLV75733, 2× TPS22917	Turns 9–36 V into 5 V and 3.3 V, charges the backup pack and switches the radios fully off when they are not needed.
Brain	ESP32-S3-WROOM-1	Runs the firmware: decides when to wake, what to send, encrypts it (TLS), BLE setup, OTA.
Cellular	SIMCom SIM7080G	LTE-M / NB-IoT data and SMS. It sleeps at 3 µA (PSM) between reports.
GNSS	u-blox NEO-F10N	Dual-band L1/L5 position from GPS, Galileo, BeiDou and the Indian **NavIC** required by AIS-140.
Motion	TDK IIM-42652	Wakes the system when the parked bike is touched; detects crashes and falls.
Memory & time	W25Q128 flash, RV-3028 RTC, LC709203F	Stores ≈ 400 000 positions offline, keeps time, measures battery charge and logs lid tamper.
I/O	J1 Micro-Fit 2×4	Ignition sense, SOS button, spare digital/analog input, starter-cut relay output.

3. Use cases

Each row is one real-life situation. “Device” is what the tracker does by itself; “People see” is what reaches the owner, fleet manager or control room.

Use case	Situation	Device	People see
Live tracking	Bike is being ridden	GNSS fix every second, reports every 10 s (configurable), speed and heading	Moving dot on a map, trip history, distance, overspeed
Theft alert	Parked bike is pushed, lifted or towed	IMU wakes the CPU without ignition → GNSS + LTE on → alarm and fast reporting	Push notification “Bike moved!” with live position
Immobiliser	Owner/police wants to stop a stolen bike	Server command → DOUT1 drives the starter-cut relay (only at low speed or standstill)	“Engine disabled” confirmation; re-enable from the app
Crash detection	Rider falls or hits something	IMU sees > 6 g shock + bike lying > 60° for > 2 s → crash event with position	Emergency alert to family / fleet / control room
Panic / SOS	Rider or passenger in danger (AIS-140)	SOS button wakes the device from any state, sends emergency packet + SMS fallback	Emergency on the state control-room screen
Geofence	Vehicle leaves an allowed zone or enters a banned one	Zones stored on the device, checked on every fix (20 m hysteresis)	Enter / exit alert; works even with no network (sent later)
Power-cut / tamper	Thief cuts the tracker wire or opens the box	Detects loss of vehicle voltage or lid switch, keeps tracking on the 18650 for ≈ 40 h	“Main power removed” / “Tamper” alert, tracking continues
Battery health	Bike left parked for weeks	Measures the bike's battery and the backup pack; deep sleep ≈ 110 µA	Low-battery warning before the bike will not start
Offline log	No coverage (tunnel, hills, basement)	Positions written to 16 MB flash, uploaded in order when the network returns	Complete route with no gaps
Remote update	Bug fix or new feature	Downloads new firmware over LTE, verifies it, swaps A/B slot, rolls back if it fails	Fleet updated without visiting the vehicles
BLE setup	Installer configures a new unit	BLE open for 5 min after power-up, phone pairing with passkey	Vehicle number, server and zones set from a phone
AIS-140 fleet	Taxi bikes, rentals, delivery and commercial 2-wheelers	NavIC fix, SOS, tamper, required packet format to the state backend	Compliance for permits and the RTO

4. Operating workflow: the device's modes

The tracker is always in one of a few modes. It spends most of its life **parked and asleep** to protect the bike's battery and wakes instantly when something happens. Arrows show what makes it change mode.

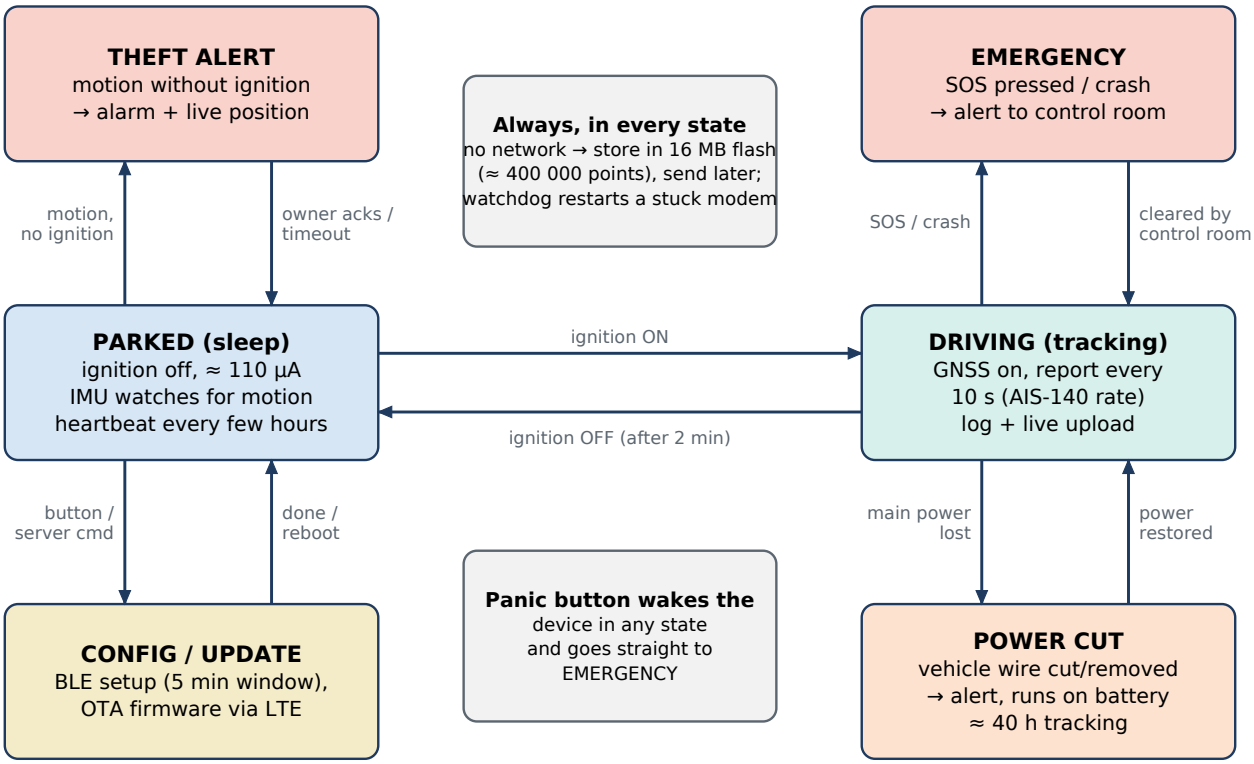


Figure 3: operating modes and transitions.

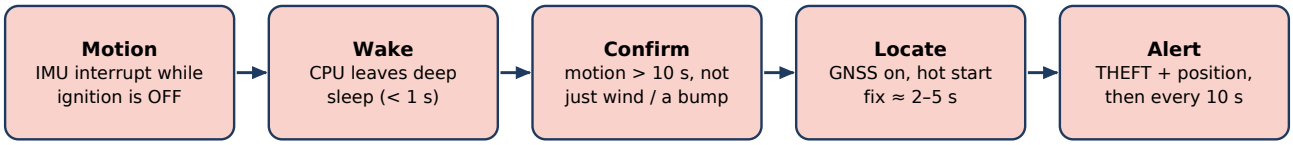
Power in each mode

Mode	What is on	Current (approx.)	Comment
Parked	LDO, RTC, IMU wake-on-motion, modem PSM, GNSS backup memory	≈ 110 µA (pack) + ≈ 25 µA (bike)	> 12 months on the 3000 mAh pack
Driving	GNSS, modem, CPU	≈ 70 mA average	from the bike; the pack charges at the same time
Power cut	same as driving, from the pack	≈ 70 mA	≈ 40 h; AIS-140 asks for ≥ 4 h
Transmit burst	modem RF	up to 0.5 A for ms	buffered by 110 µF at the modem

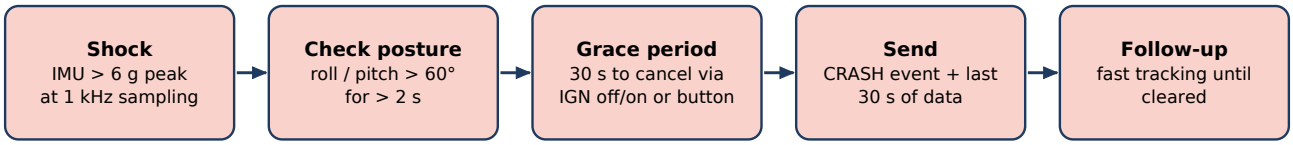
5. Step-by-step flows for the key alerts

These are the exact sequences the firmware follows. They are useful for the control-room team (what will arrive, and when) and for firmware testing.

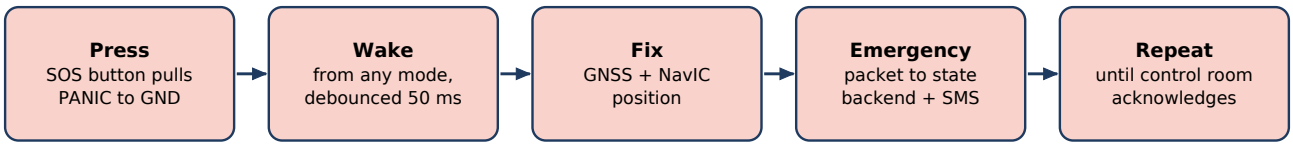
Theft: parked bike is moved



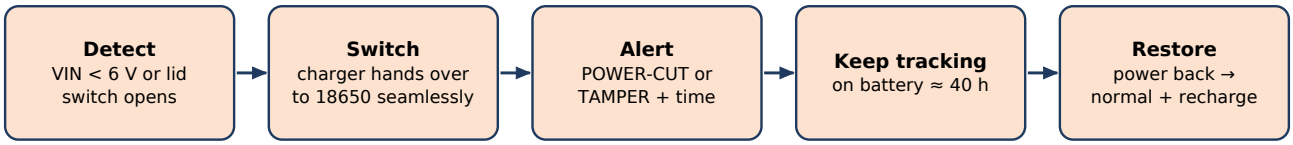
Crash / fall



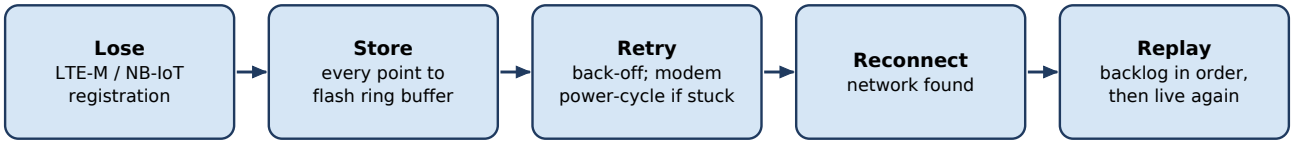
Panic / SOS (AIS-140)



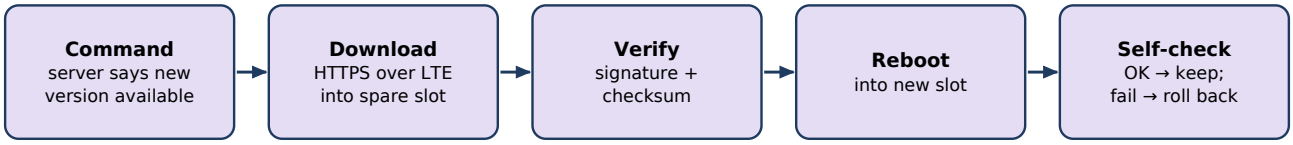
Main power cut / tamper



No network

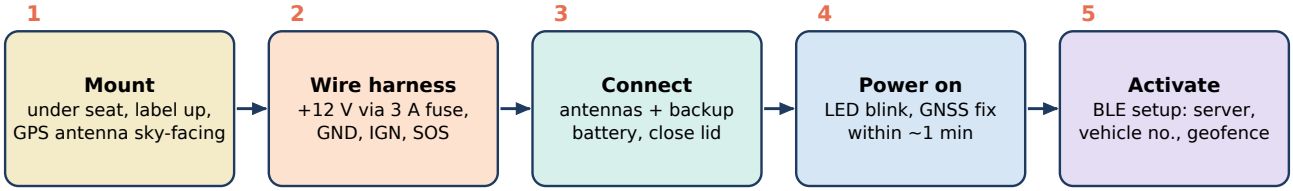


Firmware update (OTA)



6. Installation and wiring

Typical install time is 20–30 minutes. The box goes under the seat or a side panel. The GPS antenna must see the sky through plastic, not metal.



Harness connector J1 (Molex Micro-Fit 3.0, 2 × 4)

Pin	Signal	Connect to	Notes
1	+12V (BAT)	battery + through a 3 A inline blade fuse	permanent supply, 9–36 V; reverse-polarity proof
2	IGN	ignition-switched wire (key ON = 12 V)	wakes the device, starts a trip
3	PANIC / SOS	SOS push-button to GND	mandatory for AIS-140
4	DIN1	spare input (e.g. seat/lid switch, door)	active high, 9–36 V tolerant
5	GND	battery − / chassis	short, thick wire
6	AIN1	analog sensor 0–30 V (e.g. fuel level)	read by the ADC
7	DOUT1	starter-cut relay coil (low side)	immobiliser; 12 V relay, max 500 mA
8	GND	second ground	relay return / sensor ground

Other connections

Item	Detail
GPS/NavIC antenna	active L1/L5 patch on U.FL J7; mount face-up under the seat plastic or tail cowl
LTE antenna	flex antenna on U.FL J6; stick on the inner lid, ≥ 2 cm away from metal and the GPS patch
Backup battery	1S 18650 pack with protection board and 10 k NTC, on the bottom connector J2
SIM	nano-SIM holder J5 for rev A (an embedded eSIM footprint is planned for rev B, see section 10)
USB-C (J3)	service only: firmware flash, logs and setup on the bench; keep it sealed in the field

Installer checklist: fuse fitted · IGN goes high with the key · SOS triggers an alert in the app · GNSS fix shown · LTE signal OK · relay cuts the starter when commanded · lid closed and the cable gland tight (IP67).

7. Design workflow: how the PCB was made

The design is **code-first**. One Python file (design.py) describes every part and connection. Scripts turn it into the KiCad schematic, the board, and the manufacturing files. Changing a part means editing one line and re-running the scripts, so the schematic, PCB, BOM and firmware pin map never drift apart.

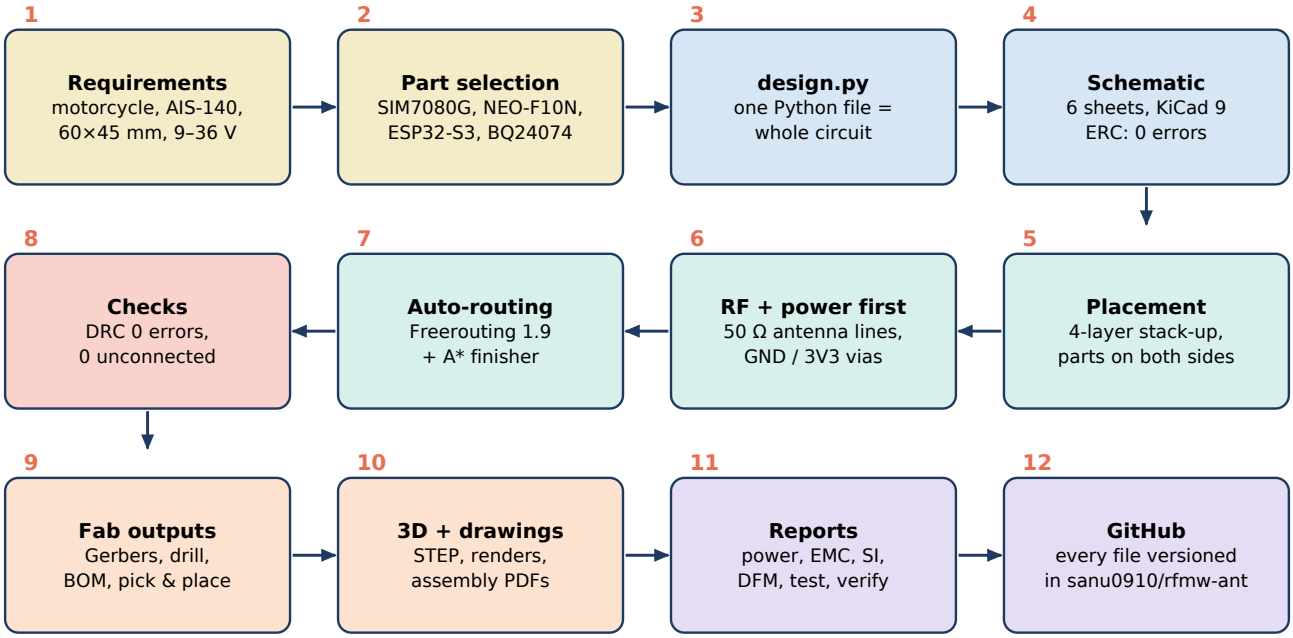


Figure 4: design pipeline, read in a snake (row 1 → right, row 2 ← left, row 3 → right).

Tools and the script for each step

Step	Tool / script	Output
Circuit	02_python/design.py, make_lib.py	parts, nets, MPNs, library symbols and footprints
Schematic	make_schematic.py → KiCad 9 (kicad-cli ERC)	tracker.kicad_sch + 6 sub-sheets, schematic PDF
Board	make_pcb.py place / dsn (pcbnew Python)	outline, 4-layer stack-up, rules, placement
RF + power	make_pcb.py, rf_calc.py	50 Ω coplanar lines to U.FL, GND & 3.3 V vias
Routing	Freerouting 1.9 + patch_route.py (A* with rip-up)	all 180 parts connected, 399 vias
Verification	kicad-cli DRC with schematic parity	0 errors, 0 unconnected, parity clean
Manufacturing	kicad-cli export, make_bom.py, make_cpl.py	Gerbers zip, JLC BOM + CPL, engineering BOM
Docs	make_models.py, make_pins.py, this script	STEP, renders, board_pins.h, reports, this PDF
Altium	Altium Import Wizard → KiCad	same design opened natively in Altium 21+
AI assistant	KiCad MCP server (/opt/KiCAD-MCP-Server)	244 tools to inspect / edit the board from Claude

8. Production and test workflow

From the Gerber zip to a sealed, labelled tracker. Every unit goes through the same ten steps. The fixture uses the bottom-side pogo pads (TP1–TP28), so no extra connector is needed.

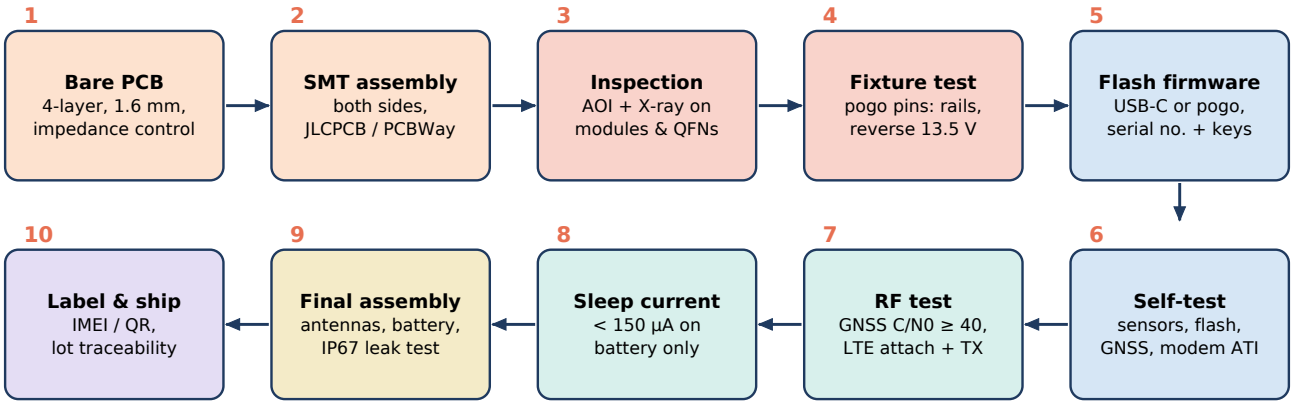
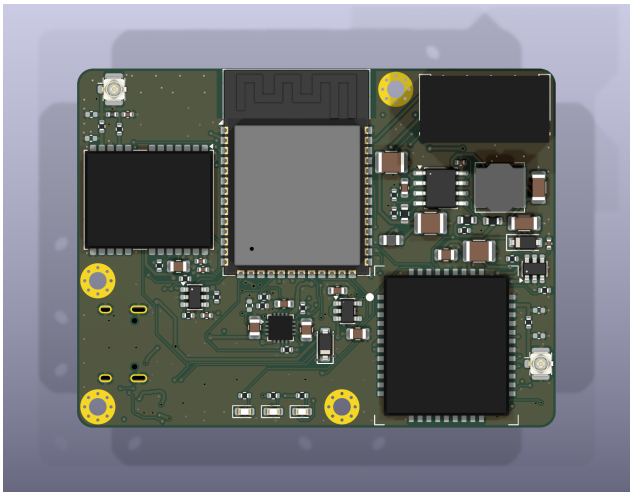


Figure 5: production flow, read in a snake (row 1 → right, row 2 ← left).

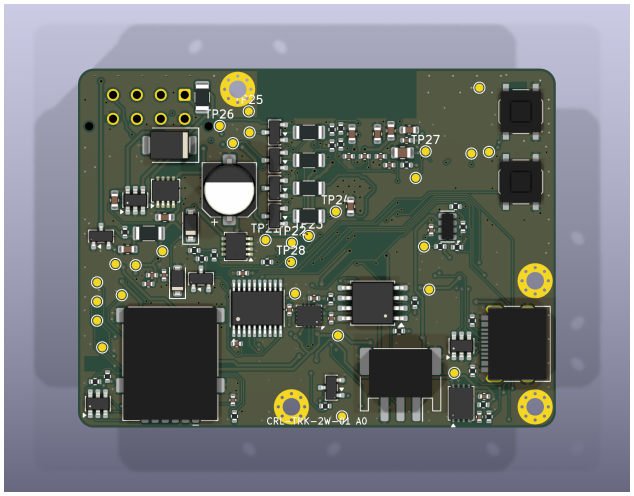
Pass / fail limits

Test	How	Pass
Power rails	13.5 V / 1 A limit on J1, probe TP10–TP16	5.0 ± 0.1 V, VSYS 4.4 V, 3.30 ± 0.05 V, input < 30 mA
Reverse / over-voltage	−13.5 V for 1 s, then 30 V	< 1 mA reverse, rails unchanged, no damage
Digital self-test	test firmware: I²C scan, flash ID, IMU ID, GNSS version, modem ATI / CCID	all devices answer
Inputs / output	fixture drives IGN, SOS, DIN1, AIN1; loads DOUT1	levels and ADC within ± 3 %
GNSS RF	−130 dBm L1 + L5 from a generator	C/N0 ≥ 40 dB-Hz
LTE RF	shielded box / call-box	network attach, RSSI and TX power within spec
Sleep	battery only, 60 s after sleep command	< 150 µA
Charger	NTC simulated at 25 °C and 50 °C	charges at 25 °C, stops at 50 °C
Enclosure	pressure-decay leak test	IP67

Board views



Top: modules, power, connectors



Bottom: small parts, SIM, USB-C, test pads

9. Which file do I need?

All files are in **projects/07_ais140-2w-gnss-tracker/** of the GitHub repository.

I am a …	I want to …	Open
Fleet owner / manager	understand features and alerts	this PDF (sections 1, 3–5)
Installer	wire and mount the unit	this PDF section 6; 04_docs/DESIGN_REPORT.md §6
Purchasing	order parts and boards	03_fab/BOM_JLCPCB.csv, BOM_engineering.csv, 04_docs/BOM.md
PCB fab / assembler	make the board	03_fab/CRL-TRK-2W-01_gerbers.zip, CPL_JLCPCB.csv, BOM_JLCPCB.csv
Hardware engineer	review or change the design	01_kicad/tracker.kicad_pro, 02_python/design.py, 04_docs/DESIGN_REPORT.md
Altium user	open the design in Altium	Import Wizard → KiCad → 01_kicad/tracker.kicad_pro
Firmware developer	write the code	06_firmware/board_pins.h, DESIGN_REPORT §10 (firmware architecture)
Mechanical designer	design the enclosure	05_renders/CRL-TRK-2W-01.step, assembly_top/bottom.pdf
Test / quality	build the fixture	DESIGN_REPORT §8 (test pads, production checklist)
Certification	plan AIS-140 / BIS / WPC / TEC	DESIGN_REPORT §9, 04_docs/VERIFY_LIST.md

Regenerate everything

```
cd projects/07_ais140-2w-gnss-tracker/02_python
python3 make_lib.py && python3 make_schematic.py && python3 make_bom.py && python3 make_pins.py
python3 make_pcb.py dsn → Freerouting 1.9 → make_pcb.py ses / finish / patch
python3 make_cpl.py && python3 make_workflow_pdf.py
```

10. Glossary

Term	Meaning
AIS-140	Indian standard for vehicle location tracking devices in public and commercial vehicles
VLTD	Vehicle Location Tracking Device: the AIS-140 name for this product
GNSS	all satellite navigation systems together (GPS, Galileo, BeiDou, NavIC ...)
NavIC / IRNSS	India's own satellite navigation system, required by AIS-140
L1 / L5	two GNSS frequency bands; using both gives better accuracy near buildings and trees
LTE-M / NB-IoT	low-power versions of 4G for small devices; good indoor coverage, low data cost
PSM	Power Saving Mode: the modem sleeps for hours but stays registered on the network
IMU	Inertial Measurement Unit: accelerometer + gyroscope that senses motion and crashes
Geofence	a virtual boundary on the map; crossing it raises an alert
Immobiliser	relay that stops the starter motor so the bike cannot be started
OTA	Over-The-Air firmware update through the mobile network
MQTT / HTTPS	the internet protocols used to send data securely to the server
BLE	Bluetooth Low Energy: used to configure the device from a phone
U.FL	tiny snap-on RF connector for the antennas
ERC / DRC	Electrical / Design Rule Check: automatic checks of the schematic and board
Gerber, BOM, CPL	board-making files, Bill of Materials, and pick-and-place component positions
IP67	dust-tight and survives immersion to 1 m for 30 minutes

Open items before mass production

Item	Why it matters	Plan
Component values	some values were set without the vendor datasheet at hand	check every line in 04_docs/VERIFY_LIST.md before ordering
Embedded SIM	AIS-140 asks for an eSIM with multi-operator profiles	add an MFF2 eSIM footprint in rev B
2G fallback	SIM7080G is LTE-M / NB-IoT only	confirm acceptance with the test agency (ICAT / ARAI)
Certification	CDAC / ICAT type approval, BIS, WPC ETA, TEC	pre-compliance tests on rev A prototypes
Firmware	the hardware is ready; the features above need implementing	follow DESIGN_REPORT §10 and board_pins.h